

数据结构 DataStructure/

约定：默认 **1-indexed**（构造传点数/元素数 n ，内部开 $n+1$ ）。依赖 `Template/ihxnan.hpp` 的类型别名（`ll`、`vi`、`pii` 等）；`SegmentTree` 还依赖宏 `ls` / `rs`。

DSU 并查集 DSU.hpp

- **用法：**DSU dsu(n) (点号 1..n); merge(x,y) 合并，返回是否真的合并（可用于 Kruskal 计数）； same(x,y) 判连通； size(x) 返回 x 所在连通块大小。
- **实现：**路径减半 + 按大小合并，均摊近 $O(\alpha)$ 。
- **注意：**下标 0 也存在（自根），但按约定从 1 开始使用。

```
struct DSU
{
    vector<int> fa, sz;

    DSU(int n) : fa(n + 1), sz(n + 1, 1)
    {
        iota(fa.begin(), fa.end(), 0); // 初始每个点自为根
    }

    // 查根（路径减半：边走边把 fa 指向祖父）
    int get(int x)
    {
        while (x != fa[x])
            x = fa[x] = fa[fa[x]];
        return x;
    }

    // 合并，返回是否合并成功
    bool merge(int x, int y)
    {
        if ((x = get(x)) == (y = get(y)))
            return false;
        fa[y] = x;
        sz[x] += sz[y]; // 按大小合并
        return true;
    }

    bool same(int x, int y)
    {
        return get(x) == get(y);
    }

    int size(int x) // x 所在连通块大小
    {
        return sz[get(x)];
    }
};
```

树状数组 Fenwick.hpp

- **用法**: `Fenwick<T> fw(n)`; `add(x, v)` 单点加; `query(x)` 前缀和 `[1..x]`; `query(l, r)` 区间和; `select(k)` 树上二分。
- **复杂度**: 单次 $O(\log n)$ 。
- **select**: 返回满足"前缀和 $\leq k$ "的最大下标 x (配合"每个位置 `add(pos, 1)`"可查第 k 小: 答案为 `select(k - 1) + 1`)。

```
template <typename T> struct Fenwick
{
    int n;
    vector<T> tree;

    Fenwick(int n) : n(n), tree(n + 1)
    {
    }

    // 单点加: pos += v
    void add(int x, const T &v)
    {
        for (; x ≤ n; x += x & -x) // x & -x 取 lowbit
            tree[x] += v;
    }

    // 前缀和 [1..x]
    T query(int x)
    {
        T ans{};
        for (; x > 0; x -= x & -x)
            ans += tree[x];
        return ans;
    }

    // 区间和 [l..r]
    T query(int l, int r)
    {
        return query(r) - query(l - 1);
    }

    // 树上二分: 返回前缀和 ≤ k 的最大 x
    int select(const T &k)
    {
        int x = 0;
        T cur{};
        for (int i = 1 << __lg(n); i; i >>= 1) // 从 2^floor(log n) 往下试
            if (x + i ≤ n && cur + tree[x + i] ≤ k)
                cur += tree[x += i];
        return x;
    }
};
```

线段树（区间加 / 区间求和） SegmentTree.hpp

- 用法：SegmentTree<ll> sg(n) 或 SegmentTree<ll> sg(arr)（arr 为 1-indexed、长度 n+1）；update(l, r, k) 区间加 k；query(l, r) 区间和；重建可调 build()。
- 复杂度：单次 $O(\log n)$ 。
- 注意：依赖骨架宏 ls / rs；T 需支持 +、+=，且懒标记 0 表示无标记；区间长用 $(r - l + 1) * k$ ，注意溢出（建议 T = ll）。

```
template <class T> struct SegmentTree
{
    int n;
    vector<T> in;          // 原数组 (1-indexed)
    vector<T> tree, tag;   // 节点值 / 懒标记

    SegmentTree(int n) : n(n), in(n + 1), tree(4 * n), tag(4 * n)
    {
    }

    SegmentTree(vector<T> &arr) : n(arr.size() - 1), in(arr), tree(4 * n), tag(4 * n)
    {
        build(); // 直接从数组建树
    }

    // 由儿子更新父亲
    void push_up(int p)
    {
        tree[p] = tree[ls(p)] + tree[rs(p)];
    }

    void build(int p, int l, int r)
    {
        if (l == r)
        {
            tree[p] = in[l];
            return;
        }
        int mid = l + r >> 1;
        build(ls(p), l, mid);
        build(rs(p), mid + 1, r);
        push_up(p);
    }

    // 给 p 覆盖的整段 [l, r] 整体加 k，并打懒标记
    void change(int p, int l, int r, T k)
    {
        tree[p] += (r - l + 1) * k;
        tag[p] += k;
    }

    void push_down(int p, int l, int r)
    {
        if (tag[p]) // 有标记才下传
        {
            int mid = l + r >> 1;
            change(ls(p), l, mid, tag[p]);
            change(rs(p), mid + 1, r, tag[p]);
            tag[p] = 0;
        }
    }

    // 递归修改：把 [ul, ur] 整体加 k
    void update(int ul, int ur, int p, int l, int r, T k)
```

```

{
    if (ul ≤ l && r ≤ ur) // 当前区间被完全覆盖
    {
        change(p, l, r, k);
        return;
    }
    push_down(p, l, r);
    int mid = l + r >> 1;
    if (ul ≤ mid)
        update(ul, ur, ls(p), l, mid, k);
    if (ur > mid)
        update(ul, ur, rs(p), mid + 1, r, k);
    push_up(p);
}

// 递归查询 [ql, qr] 的和
T query(int ql, int qr, int p, int l, int r)
{
    if (ql ≤ l && r ≤ qr)
        return tree[p];
    push_down(p, l, r);
    T ans = 0;
    int mid = l + r >> 1;
    if (ql ≤ mid)
        ans += query(ql, qr, ls(p), l, mid);
    if (qr > mid)
        ans += query(ql, qr, rs(p), mid + 1, r);
    return ans;
}

// ---- 对外接口（以整棵树为范围） ----
void build()
{
    build(1, 1, n);
}

void update(int ul, int ur, T k) // [ul, ur] 区间加 k
{
    update(ul, ur, 1, 1, n, k);
}

T query(int ql, int qr) // 查询 [ql, qr] 区间和
{
    return query(ql, qr, 1, 1, n);
}
};

```

ST 表（稀疏表） ST.hpp

- **用法**：ST st(n) 或 ST st(arr) (1-indexed、长度 $n+1$)；先 init()，再 query(l,r)。
- **复杂度**：预处理 $O(n \log n)$ ，查询 $O(1)$ 。
- **注意**：仅适用于**可重复结合**（幂等）运算，默认取 max，可传自定义二元函数（如 gcd）；只支持 int。

```
struct ST
{
    const int n, len;
    vector<int> in;
    vector<vector<int>> st;          // st[i][j]: [j, j+2^i-1] 的答案
    function<int(int, int)> Max;    // 可重复结合的合并函数

    ST(
        int n, function<int(int, int)> Max = [](int a, int b) { return max(a, b); })
        : n(n), len(__lg(n) + 1), in(n + 1), Max(Max)
    {
        st.resize(len, vector<int>(n + 1));
    }

    ST(
        vector<int> &arr, function<int(int, int)> Max = [](int a, int b) { return max(a, b); })
        : n(arr.size() - 1), len(__lg(n) + 1), in(arr), Max(Max)
    {
        st.resize(len, vector<int>(n + 1));
    }

    // 预处理：倍增递推
    void init()
    {
        for (int i = 1; i <= n; ++i)
            st[0][i] = in[i];
        for (int i = 1; i < len; ++i)
            for (int j = 1; j + (1 << i) - 1 <= n; ++j)
                st[i][j] = Max(st[i - 1][j], st[i - 1][j + (1 << i - 1)]);
    }

    // O(1) 查询 [l, r]: 两个恰好覆盖的区间取 Max
    int query(int l, int r)
    {
        int k = __lg(r - l + 1);
        return Max(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

整数位封装 Bit.hpp

- 用法：bit<unsigned long long> b(x)；把整数的位运算封装成成员函数 / 运算符，可读性更好。
- 注意：位下标从 LSB（第 0 位）开始；T 需为无符号整型（unsigned / unsigned long long 等），避免右移补符号位。小规模（位数少）直接裸位运算更快。

```
template <class T> struct bit
{
    T value;
    bit(T x = 0) : value(x) {}
    bool operator[](int x) const { return value >> x & 1; } // 读第 x 位
    void set(int x) { value |= T(1) << x; } // 置 1
    void reset(int x) { value &= ~(T(1) << x); } // 置 0
    void flip(int x) { value ^= T(1) << x; } // 翻转
    bit operator<<(int x) const { return bit(value << x); }
    bit operator>>(int x) const { return bit(value >> x); }
    bit &operator<=(int x) { value <= x; return *this; }
    bit &operator>=(int x) { value >= x; return *this; }
    bit operator&(const bit &o) const { return bit(value & o.value); }
    bit operator|(const bit &o) const { return bit(value | o.value); }
    bit operator^(const bit &o) const { return bit(value ^ o.value); }
    bit &operator&=(const bit &o) { value &= o.value; return *this; }
    bit &operator|=(const bit &o) { value |= o.value; return *this; }
    bit &operator^=(const bit &o) { value ^= o.value; return *this; }
    bit operator~() const { return bit(~value); }
    bool operator==(const bit &o) const { return value == o.value; }
    bool operator!=(const bit &o) const { return value != o.value; }
    bool operator<(const bit &o) const { return value < o.value; }
    bool operator>(const bit &o) const { return value > o.value; }
    bool operator<=(const bit &o) const { return value <= o.value; }
    bool operator>=(const bit &o) const { return value >= o.value; }
};
```

FHQ Treap (数组版, 按值 / 按大小分裂) Treap.hpp

- **用法:** `Treap<int> tp`(节点数上限); `insert(x)` 插入; `erase(x)` 删除一个 `x`; `rank(x)` 比 `x` 小的元素个数 + 1; `val(k)` 第 `k` 小; `pre(x)` 前驱 (`< x` 的最大值); `back(x)` 后继 (`> x` 的最小值); `count(x)` `x` 的个数; `size()` 元素总数。
- **复杂度:** 单次期望 $O(\log n)$ 。
- **注意:** 构造参数为**节点数上限** (内部开 `n+1`, 每次 `insert` 新建一个节点); `split_val` 的 `eq` 参数控制 `== k` 归左还是归右; 指针版见 `TreapPtr`。

```
template <class T> struct Treap
{
    struct Node
    {
        T val;
        int key, siz, ch[2]; // key: 随机优先级 (大根堆), siz: 子树大小
        void clear()
        {
            ch[0] = ch[1] = siz = key = 0;
            val = T{};
        }
    };

    int cnt, root;
    vector<Node> t;
    mt19937_64 gen;

    Treap(int n) : cnt(0), root(0), t(n + 1), gen(chrono::steady_clock::now().time_since_epoch().count())
    {
    }

    int rnd()
    {
        return int(gen());
    }

    // 用儿子更新自己的子树大小
    void update(int now)
    {
        t[now].siz = t[t[now].ch[0]].siz + t[t[now].ch[1]].siz + 1;
    }

    // 合并两棵树 (u 中所有值 ≤ v), 按 key 大根堆性质
    int merge(int u, int v)
    {
        if (!u || !v)
            return u | v;
        if (t[u].key < t[v].key)
        {
            t[u].ch[1] = merge(t[u].ch[1], v);
            update(u);
            return u;
        }
        else
        {
            t[v].ch[0] = merge(u, t[v].ch[0]);
            update(v);
            return v;
        }
    }

    // 按值分裂: < k (eq 时含 = k) 的进 x, 其余进 y
    void split_val(int now, const T &k, int &x, int &y, bool eq = true)
```

```

{
    if (!now)
        return (void)(x = y = 0);
    if (t[now].val < k || (eq && t[now].val == k))
        x = now, split_val(t[now].ch[1], k, t[now].ch[1], y, eq);
    else
        y = now, split_val(t[now].ch[0], k, x, t[now].ch[0], eq);
    update(now);
}

// 按名次分裂：前 k 小进 x
void split_k(int now, int k, int &x, int &y)
{
    if (!now)
        return (void)(x = y = 0);
    if (t[t[now].ch[0]].siz < k)
        x = now, split_k(t[now].ch[1], k - t[t[now].ch[0]].siz - 1, t[now].ch[1], y);
    else
        y = now, split_k(t[now].ch[0], k, x, t[now].ch[0]);
    update(now);
}

// 取第 k 小（并合并回 now）
T kth(int &now, int k)
{
    int a, b, c, d;
    split_k(now, k - 1, a, b);
    split_k(b, 1, c, d);
    T res = t[c].val;
    now = merge(a, merge(c, d));
    return res;
}

void insert(const T &x) // 新建节点，按值分裂后插入中间
{
    int u = ++cnt, a, b;
    t[u].val = x, t[u].key = rnd(), t[u].siz = 1;
    split_val(root, x, a, b);
    root = merge(merge(a, u), b);
}

void erase(const T &x) // 删除一个 x（不存在则无操作）
{
    int a, b, c, d;
    split_val(root, x, a, b, false); // a: < x
    split_k(b, 1, c, d);             // c: 值为 x 的一个节点
    if (c && t[c].val == x)
        t[c].clear(), root = merge(a, d);
    else
        root = merge(a, merge(c, d));
}

int size() // 元素总数
{
    return t[root].siz;
}

int count(const T &x) // x 出现次数
{
    int a, b, c, d;
    split_val(root, x, a, b, false); // a: < x
    split_val(b, x, c, d, true);     // c: == x
    int res = t[c].siz;
    root = merge(a, merge(c, d));
}

```

```

        return res;
    }

    int rank(const T &x) // x 的排名 (比它小的个数 + 1)
    {
        int a, b;
        split_val(root, x, a, b, false);
        int res = t[a].siz + 1;
        root = merge(a, b);
        return res;
    }

    T val(int k) // 第 k 小
    {
        return kth(root, k);
    }

    T pre(const T &x) // 前驱 (< x 的最大值)
    {
        int a, b;
        split_val(root, x, a, b, false);
        T res = kth(a, t[a].siz);
        root = merge(a, b);
        return res;
    }

    T back(const T &x) // 后继 (> x 的最小值)
    {
        int a, b;
        split_val(root, x, a, b);
        T res = kth(b, 1);
        root = merge(a, b);
        return res;
    }
};

```

FHQ Treap (指针版, 免预估节点数) TreapPtr.hpp

- **用法:** 与 Treap.hpp 完全一致的 API; TreapPtr<int> tp; 直接用, 无需估计节点数上限。
- **实现:** 内存池 pool 统一管理新节点, 删除的节点进回收站 trash 复用, 析构时统一释放。
- **注意:** 禁用了拷贝 (避免双重释放); kth 对空树返回 T{}。

```
template <class T> struct TreapPtr
{
    struct Node
    {
        T val;
        int key, siz;
        Node *ch[2];
        Node(const T &v = T{}, int k = 0) : val(v), key(k), siz(1)
        {
            ch[0] = ch[1] = nullptr;
        }
    };

    Node *root;
    vector<Node *> pool, trash; // 内存池 / 回收站
    mt19937_64 gen;

    TreapPtr() : root(nullptr), gen(chrono::steady_clock::now().time_since_epoch().count()) {}
    TreapPtr(const TreapPtr &) = delete; // 禁止拷贝
    TreapPtr &operator=(const TreapPtr &) = delete;
    ~TreapPtr() // 析构统一释放内存池
    {
        for (Node *p : pool)
            delete p;
    }

    int rnd()
    {
        return int(gen());
    }

    int sz(Node *p) // 空节点大小为 0
    {
        return p ? p->siz : 0;
    }

    // 取新节点: 优先复用回收站
    Node *newNode(const T &v)
    {
        Node *p;
        if (!trash.empty())
        {
            p = trash.back(), trash.pop_back();
            p->val = v, p->key = rnd(), p->siz = 1;
            p->ch[0] = p->ch[1] = nullptr;
        }
        else
            pool.push_back(p = new Node(v, rnd()));
        return p;
    }

    void update(Node *now)
    {
        now->siz = sz(now->ch[0]) + sz(now->ch[1]) + 1;
    }
}
```

```

Node *merge(Node *u, Node *v) // 按 key 大根堆性质合并
{
    if (!u || !v)
        return u ? u : v;
    if (u->key < v->key)
    {
        u->ch[1] = merge(u->ch[1], v);
        update(u);
        return u;
    }
    else
    {
        v->ch[0] = merge(u, v->ch[0]);
        update(v);
        return v;
    }
}

void split_val(Node *now, const T &k, Node *&x, Node *&y, bool eq = true) // 按值分裂
{
    if (!now)
        return (void)(x = y = nullptr);
    if (now->val < k || (eq && now->val == k))
        x = now, split_val(now->ch[1], k, now->ch[1], y, eq);
    else
        y = now, split_val(now->ch[0], k, x, now->ch[0], eq);
    update(now);
}

void split_k(Node *now, int k, Node *&x, Node *&y) // 按名次分裂
{
    if (!now)
        return (void)(x = y = nullptr);
    if (sz(now->ch[0]) < k)
        x = now, split_k(now->ch[1], k - sz(now->ch[0]) - 1, now->ch[1], y);
    else
        y = now, split_k(now->ch[0], k, x, now->ch[0]);
    update(now);
}

T kth(Node *&now, int k) // 第 k 小
{
    Node *a, *b, *c, *d;
    split_k(now, k - 1, a, b);
    split_k(b, 1, c, d);
    T res = c ? c->val : T{};
    now = merge(a, merge(c, d));
    return res;
}

void insert(const T &x) // 插入
{
    Node *a, *b;
    split_val(root, x, a, b);
    root = merge(merge(a, newNode(x)), b);
}

void erase(const T &x) // 删除一个 x, 节点进回收站
{
    Node *a, *b, *c, *d;
    split_val(root, x, a, b, false);
    split_k(b, 1, c, d);
    if (c && c->val == x)
        trash.push_back(c), root = merge(a, d);
}

```

```

        else
            root = merge(a, merge(c, d));
    }

    int size() // 元素总数
    {
        return sz(root);
    }

    int count(const T &x) // x 出现次数
    {
        Node *a, *b, *c, *d;
        split_val(root, x, a, b, false);
        split_val(b, x, c, d, true);
        int res = sz(c);
        root = merge(a, merge(c, d));
        return res;
    }

    int rank(const T &x) // x 的排名
    {
        Node *a, *b;
        split_val(root, x, a, b, false);
        int res = sz(a) + 1;
        root = merge(a, b);
        return res;
    }

    T val(int k) // 第 k 小
    {
        return kth(root, k);
    }

    T pre(const T &x) // 前驱
    {
        Node *a, *b;
        split_val(root, x, a, b, false);
        T res = kth(a, sz(a));
        root = merge(a, b);
        return res;
    }

    T back(const T &x) // 后继
    {
        Node *a, *b;
        split_val(root, x, a, b);
        T res = kth(b, 1);
        root = merge(a, b);
        return res;
    }
};

```